

新功能更新日志

本文档记录了 AI Search Engine 项目最近添加的新功能和改进。

1. LLM 流式输出基础设施

修改文件

- src/llm/manager.py
- src/llm/openai_client.py
- src/llm/ollama_client.py

功能描述

为所有 LLM 客户端添加了流式输出支持，实现实时响应显示。

主要改动

```
# LLManager 新增方法
async def stream_complete(self, prompt: str, ...) -> AsyncGenerator[str, None]:
    """流式生成文本，逐字返回"""

def get_provider_order(self) -> List[str]:
    """获取当前提供者优先级顺序"""
```

使用示例

```
async for chunk in llm_manager.stream_complete("你好"):
    print(chunk, end="", flush=True)
```

2. Agent 流式方法

修改文件

- src/agents/chat_agent.py
- src/agents/research_agent.py
- src/agents/code_agent.py
- src/agents/rag_agent.py

功能描述

所有 Agent 现在支持流式输出，用户可以实时看到生成过程。

主要改动

```
# ChatAgent
async def stream_chat(self, message: str) -> AsyncGenerator[str, None]

# ResearchAgent
async def stream_research(self, query: str) -> AsyncGenerator[str, None]

# CodeAgent
async def stream_solve(self, problem: str) -> AsyncGenerator[str, None]

# RAGAgent
async def stream_query(self, query: str) -> AsyncGenerator[str, None]
```

3. SearchTool 缓存机制

修改文件

- src/tools/search.py

功能描述

为搜索工具添加 LRU + TTL 缓存，减少重复 API 调用，提高响应速度。

主要改动

```
class SearchCache:
    """LRU 缓存, 支持 TTL 过期"""
    def __init__(self, max_size: int = 100, ttl: int = 3600):
        self.max_size = max_size # 最大缓存条目
        self.ttl = ttl           # 过期时间 (秒)

class SearchTool:
    # 新增方法
    def clear_cache(self) -> None
    def get_cache_stats(self) -> Dict[str, Any]
    def set_cache_enabled(self, enabled: bool) -> None
```

使用示例

```
search_tool = SearchTool(api_key="xxx", cache_enabled=True, cache_ttl=3600)

# 第一次搜索 - 调用 API
results = await search_tool.search("Python 教程")

# 第二次相同搜索 - 从缓存读取
results = await search_tool.search("Python 教程")

# 查看缓存统计
stats = search_tool.get_cache_stats()
# {'size': 1, 'max_size': 100, 'hits': 1, 'misses': 1, 'hit_rate': 0.5}
```

4. HybridRouter 反馈学习机制

修改文件

- src/routing/hybrid_router.py

功能描述

路由器现在可以记录用户反馈, 自动学习和调整路由阈值, 提高路由准确性。

主要改动

```
@dataclass
class RoutingFeedback:
    """路由反馈记录"""
    query: str
    routed_task: TaskType
    correct_task: Optional[TaskType]
    is_correct: bool
    timestamp: float
    user_comment: Optional[str]

class RoutingFeedbackTracker:
    """反馈追踪器"""
    def record_feedback(self, query, routed_task, correct_task, is_correct,
user_comment)
    def get_accuracy(self, task_type: Optional[TaskType] = None) -> float
    def should_adjust_threshold(self) -> Tuple[bool, float]
    def get_feedback_stats(self) -> Dict[str, Any]
```

使用示例

```
router = HybridRouter(config, llm_manager)

# 记录正确路由
router.record_feedback("天气怎么样", TaskType.DOMAIN_WEATHER, is_correct=True)

# 记录错误路由并纠正
router.record_feedback(
    "计算斐波那契",
    TaskType.CHAT,          # 错误路由到聊天
    correct_task=TaskType.CODE, # 应该是代码
    is_correct=False
)

# 获取准确率
accuracy = router.get_routing_accuracy() # 0.5
```

5. ChatAgent 历史压缩

修改文件

- src/agents/chat_agent.py

功能描述

当对话历史过长时，自动使用 LLM 压缩旧消息为摘要，节省 token 同时保留上下文。

主要改动

```
class ChatAgent:
    def __init__(self, ...,
                 enable_compression: bool = True,
                 compression_threshold: int = 20):
        self._compressed_summary: Optional[str] = None

    async def _compress_history(self) -> None:
        """将旧对话压缩为摘要"""

    def _build_messages(self) -> List[Dict[str, str]]:
        """构建消息列表，包含压缩摘要"""

    def get_history_stats(self) -> Dict[str, Any]:
        """获取历史统计信息"""

    def set_compression_enabled(self, enabled: bool) -> None:
        """开启/关闭压缩功能"""
```

工作原理

原始历史 (20+ 条消息):
[msg1, msg2, msg3, ..., msg18, msg19, msg20]
 ↓ 压缩
压缩后:
[摘要: "用户讨论了...", msg15, msg16, msg17, msg18, msg19, msg20]

6. VectorStore 混合检索

修改文件

- src/tools/vector_store.py

功能描述

结合语义相似度和关键词匹配的混合检索，提高检索准确性。

主要改动

```
class VectorStore:
    def hybrid_search(
        self,
        query: str,
        k: int = 5,
        semantic_weight: float = 0.7, # 语义权重
        keyword_weight: float = 0.3 # 关键词权重
    ) -> List[Dict[str, Any]]:
        """混合检索：语义 + 关键词"""

    def _tokenize(self, text: str) -> List[str]:
        """分词（支持中英文）"""

    def _compute_keyword_score(self, query_tokens, doc_tokens) -> float:
        """计算关键词匹配分数（BM25 风格）"""
```

使用示例

```
vector_store = VectorStore()

# 纯语义搜索
results = vector_store.search("机器学习算法", k=5)

# 混合搜索（推荐）
results = vector_store.hybrid_search(
    "机器学习算法",
    k=5,
    semantic_weight=0.7,
    keyword_weight=0.3
)
```

7. TaskTracker 任务追踪器

新增文件

- src/workflow/task_tracker.py

修改文件

- src/workflow/__init__.py
- src/agents/master_agent.py

功能描述

提供工作流和任务的实时进度追踪，支持回调订阅。

主要类

```
class TaskStatus(Enum):
    PENDING = "pending"
    RUNNING = "running"
    COMPLETED = "completed"
    FAILED = "failed"
    CANCELLED = "cancelled"

@dataclass
class TaskProgress:
    task_id: str
    task_name: str
    status: TaskStatus
    progress: float # 0.0 - 1.0
    message: str
    duration: Optional[float]

@dataclass
class WorkflowProgress:
    workflow_id: str
    workflow_name: str
    tasks: List[TaskProgress]
    overall_progress: float
    completed_tasks: int
    failed_tasks: int

class TaskTracker:
    def create_workflow(workflow_id, workflow_name, task_names) -> WorkflowProgress
    def start_workflow(workflow_id) -> None
    def start_task(workflow_id, task_index, message) -> None
    def update_task_progress(workflow_id, task_index, progress, message) -> None
    def complete_task(workflow_id, task_index, result, message) -> None
```

```
def fail_task(workflow_id, task_index, error, message) -> None
def complete_workflow(workflow_id, success) -> WorkflowProgress
def subscribe_workflow(callback) -> None
def subscribe_task(callback) -> None
```

使用示例

```
tracker = TaskTracker()

# 创建工作流
workflow = tracker.create_workflow(
    "wf_001",
    "研究任务",
    ["搜索", "分析", "总结"]
)

# 订阅进度更新
def on_progress(workflow):
    print(f"进度: {workflow.overall_progress:.0%}")

tracker.subscribe_workflow(on_progress)

# 执行任务
tracker.start_workflow("wf_001")
tracker.start_task("wf_001", 0, "正在搜索...")
tracker.update_task_progress("wf_001", 0, 0.5, "搜索中...")
tracker.complete_task("wf_001", 0, results, "搜索完成")
```

8. WorkflowEngine 检查点恢复

修改文件

- src/workflow/workflow_engine.py
- src/workflow/__init__.py

功能描述

支持长时间运行工作流的检查点保存和恢复，防止中断后丢失进度。

主要改动

```
@dataclass
class WorkflowCheckpoint:
    workflow_id: str
    workflow_name: str
    completed_tasks: List[str]
    task_results: Dict[str, Any]
    current_task_index: int
    created_at: float
    metadata: Dict[str, Any]

class WorkflowEngine:
    def __init__(self, ...,
                 enable_checkpoints: bool = True,
                 checkpoint_dir: str = "./data/checkpoints"):
        ...

    def save_checkpoint(self, workflow: Workflow) -> Optional[str]:
        """保存检查点到文件"""

    def load_checkpoint(self, workflow_id: str) -> Optional[WorkflowCheckpoint]:
        """加载检查点"""

    def delete_checkpoint(self, workflow_id: str) -> bool:
        """删除检查点"""

    def list_checkpoints(self) -> List[str]:
        """列出所有检查点"""

    async def resume_workflow(
        self,
        workflow: Workflow,
        checkpoint: WorkflowCheckpoint
    ) -> WorkflowResult:
        """从检查点恢复工作流"""
```

使用示例

```
engine = WorkflowEngine(enable_checkpoints=True)

# 执行工作流
workflow = create_workflow(...)
result = await engine.execute(workflow)

# 如果中断, 从检查点恢复
checkpoint = engine.load_checkpoint("wf_001")
if checkpoint:
    result = await engine.resume_workflow(workflow, checkpoint)
```

测试覆盖

所有新功能都有完整的单元测试，位于 tests/test_new_features.py:

功能模块	测试数量	状态
LLM 流式	5	✓
Agent 流式	5	✓
SearchTool 缓存	9	✓
HybridRouter 反馈	7	✓
ChatAgent 压缩	6	✓
VectorStore 混合检索	4	✓
TaskTracker	10	✓
WorkflowEngine 检查点	9	✓
集成测试	3	✓
总计	58	✓

运行测试:

```
pytest tests/test_new_features.py -v
```

更新日期

2025-12-02